

Lecture 7: Language model

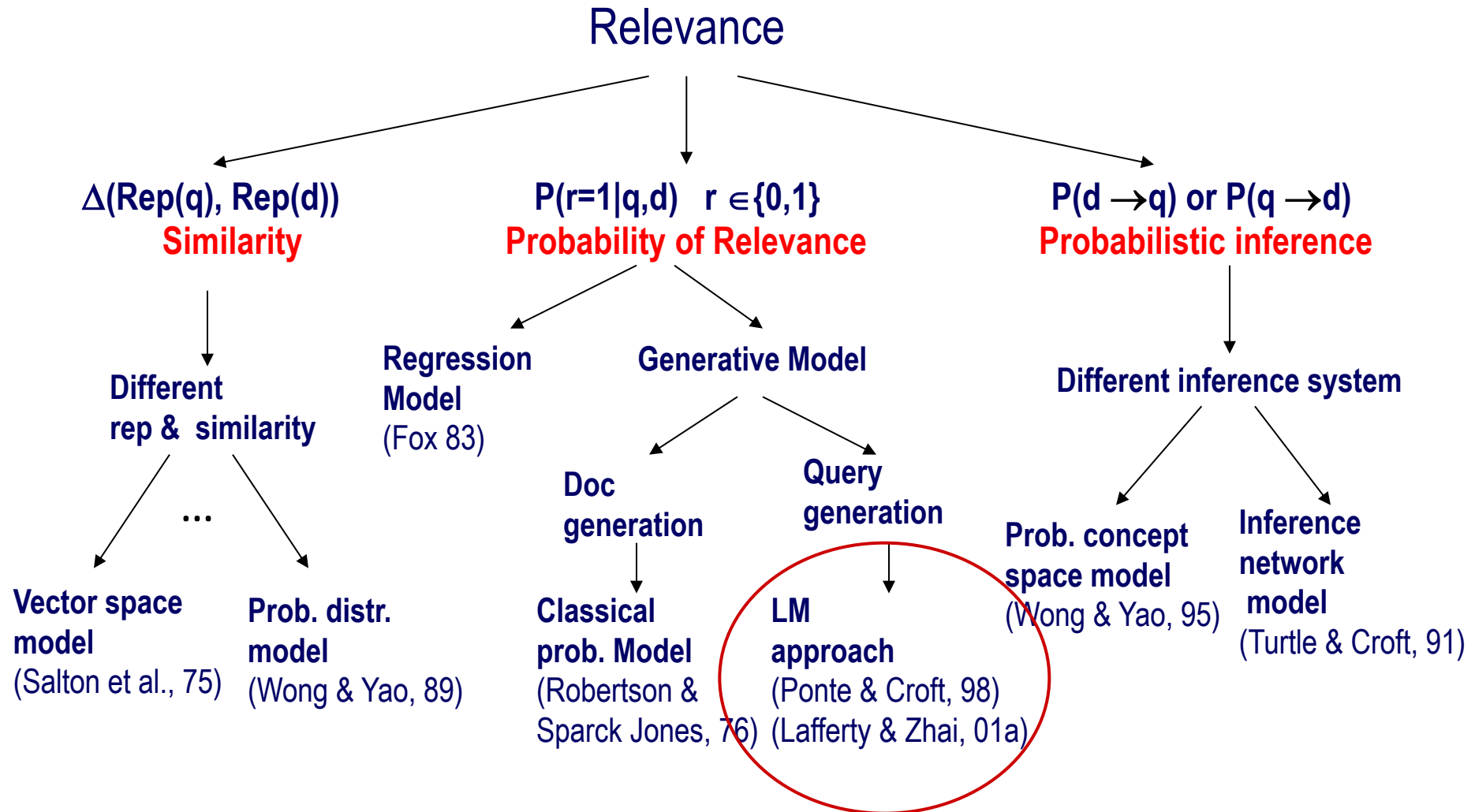
Huazheng Wang

EECS@OSU

Recap: Vector space model

- Boolean model
- Vector space model
 - Basic idea
 - TF-IDF
 - Similarity measure: Euclidean distance vs cosine similarity
- Probabilistic ranking principle
 - Essential concepts in probability
 - Justification of ranking by relevance

Notion of relevance



What is a statistical LM?

- A model specifying a probability distribution over word sequences
 - $p(\text{"*Today is Wednesday*"}) \approx 0.001$
 - $p(\text{"*Today Wednesday is*"}) \approx 0.0000000000000001$
 - $p(\text{"*The eigenvalue is positive*"}) \approx 0.00001$
- It can be regarded as a probabilistic mechanism for “generating” text, thus also called a “generative” model
 - LLMs such as GPT
- Large Language Models (LLM): still LM, but larger

Why is a LM useful?

- Provides a principled way to quantify the uncertainties associated with natural language
- Allows us to answer questions like:
 - Given that we see “*John*” and “*feels*”, how likely will we see “*happy*” as opposed to “*habit*” as the next word?
(speech recognition)
 - Given that we observe “baseball” three times and “game” once in a news article, how likely is it about “sports”?
(text categorization, information retrieval)
 - Given that a user is interested in sports news, how likely would the user use “baseball” in a query?
(information retrieval)

Language model for text

- Probability distribution over word sequences

- $p(w_1 w_2 \dots w_n) = p(w_1)p(w_2|w_1)p(w_3|w_1, w_2) \dots p(w_n|w_1, w_2, \dots, w_{n-1})$

- Complexity - $O(V^{n^*})$

- n^* - maximum ~~document~~ length sentence

We need independence assumptions!

Chain rule: from conditional probability to joint probability

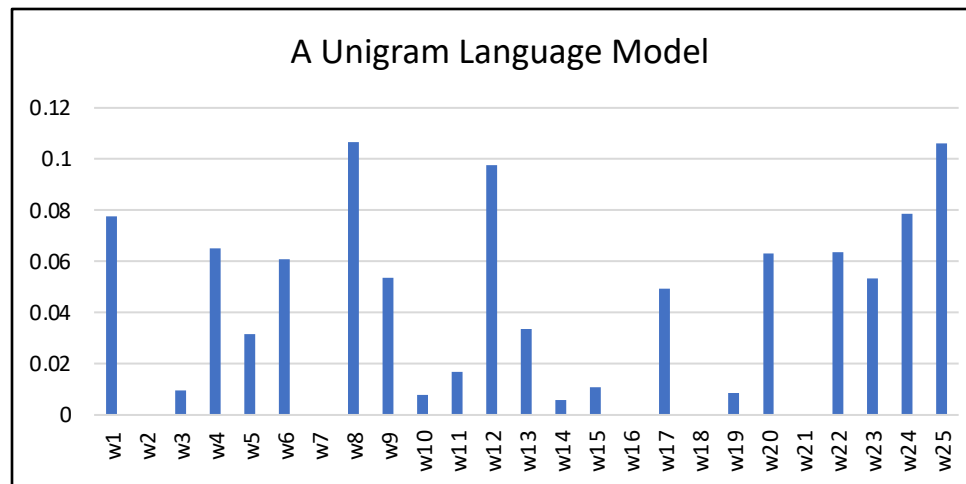
- 475,000 main headwords in Webster's Third New International Dictionary
 - Average English sentence length is 14.3 words
 - A rough estimate: $O(475000^{14})$

How large is this? $\frac{475000^{14}}{8\text{bytes} \times (1024)^4} \approx 3.38e^{66}TB$

Unigram language model

- Generate a piece of text by generating each word independently
 - $p(w_1 w_2 \dots w_n) = p(w_1)p(w_2) \dots p(w_n)$
 - $s.t. \{p(w_i)\}_{i=1}^N, \sum_i p(w_i) = 1, p(w_i) \geq 0$
- Essentially a multinomial distribution over the vocabulary

*The simplest and
most popular choice!*



More sophisticated LMs

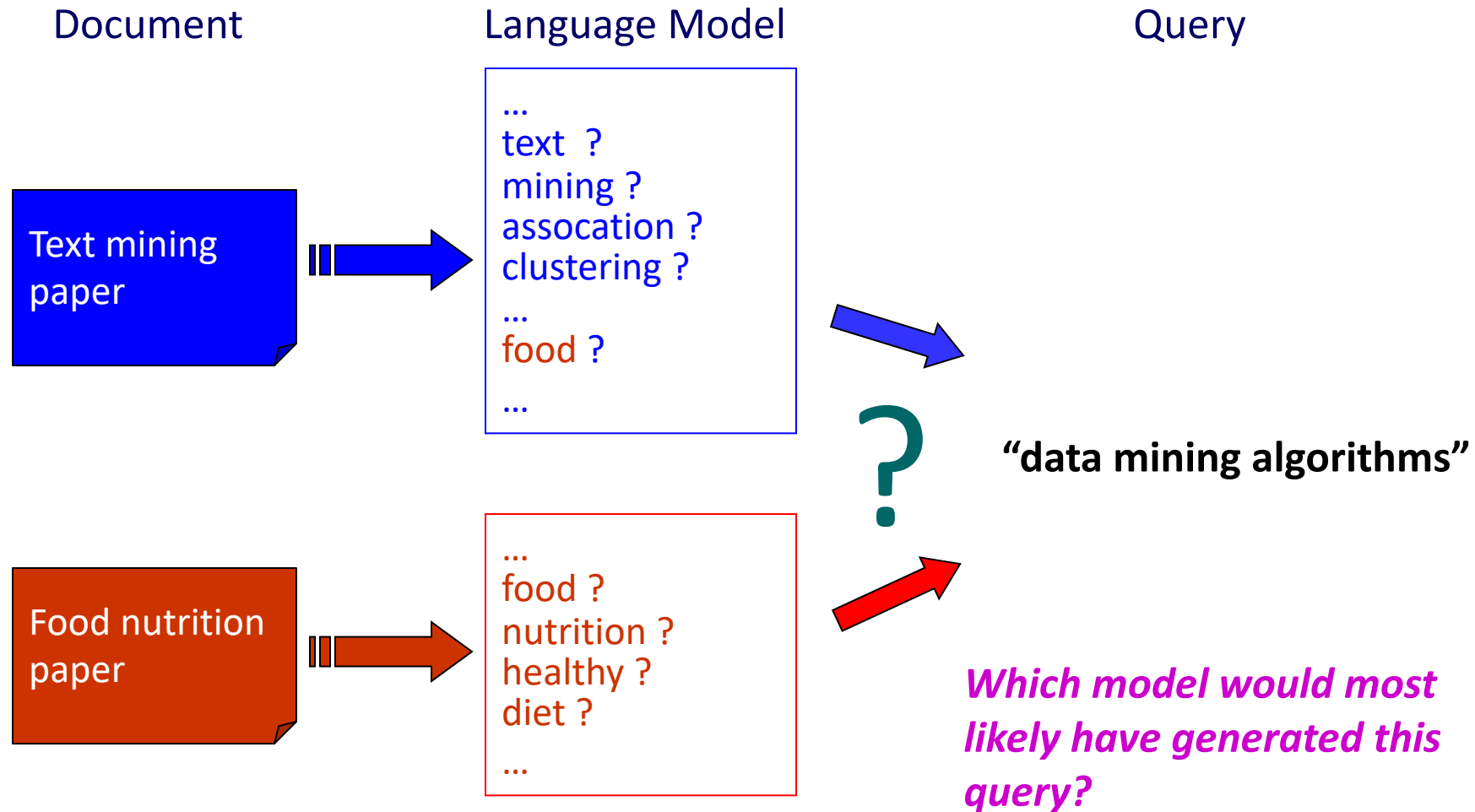
- N-gram language models
 - In general, $p(w_1 w_2 \dots w_n) = p(w_1)p(w_2|w_1) \dots p(w_n|w_1 \dots w_{n-1})$
 - N-gram: conditioned only on the past N-1 words
 - E.g., bigram: $p(w_1 \dots w_n) = p(w_1)p(w_2|w_1) p(w_3|w_2) \dots p(w_n|w_{n-1})$
- Remote-dependence language models (e.g., Maximum Entropy model)
- Structured language models (e.g., probabilistic context-free grammar)

Why just unigram models?

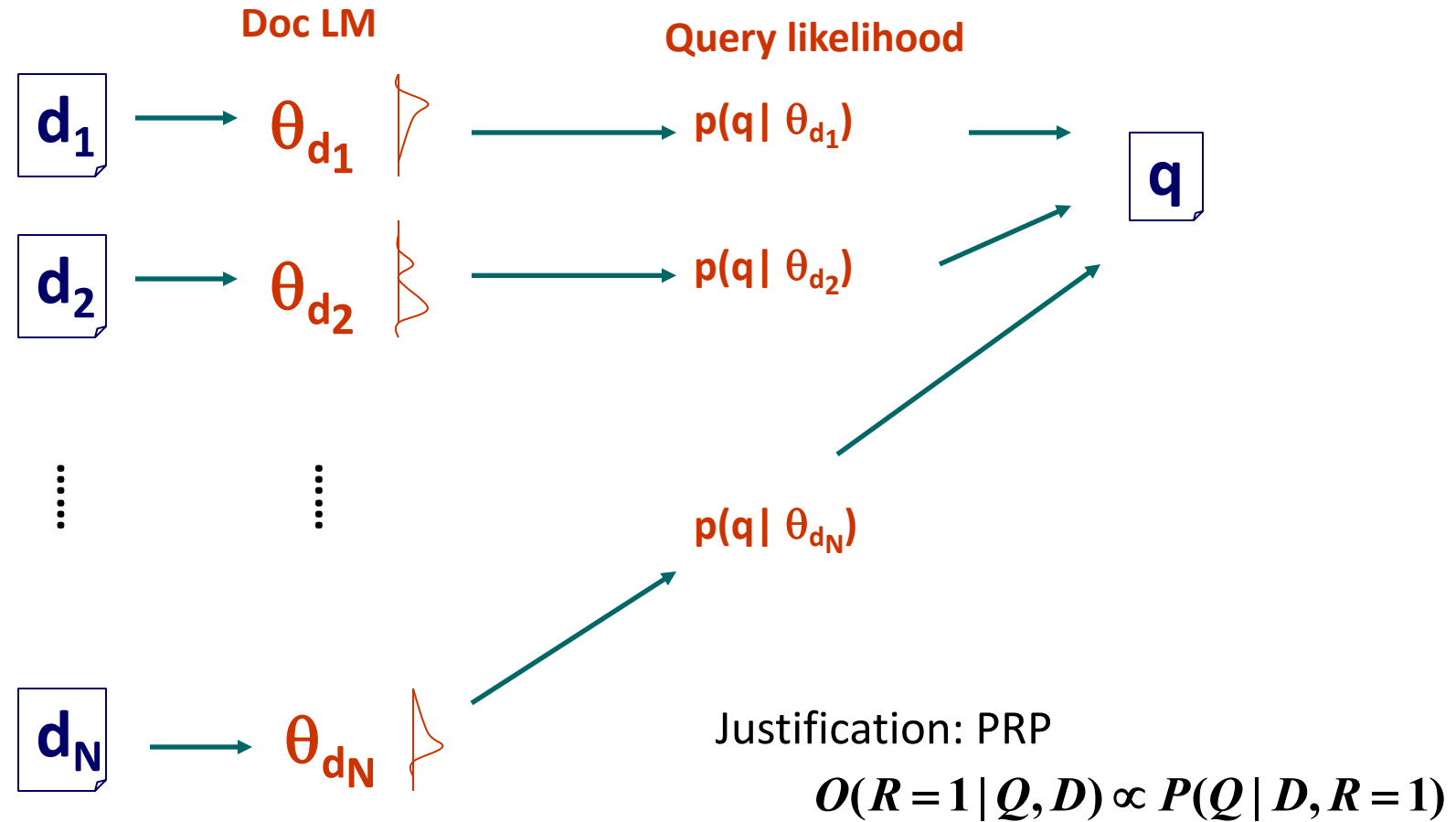
- Difficulty in moving toward more complex models
 - They involve more parameters, so need more data to estimate
 - They increase the computational complexity significantly, both in time and space
- Capturing word order or structure may not add so much value for “topical inference”
- But, using more sophisticated models can still be expected to improve performance ...

Language models for IR

[Ponte & Croft SIGIR'98]



Ranking docs by query likelihood



Retrieval as language model estimation

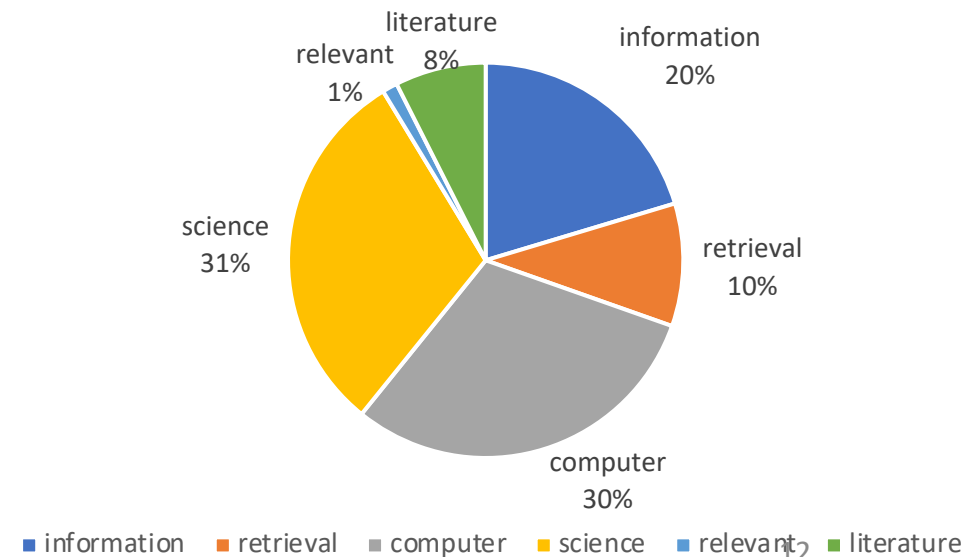
- Document ranking based on *query likelihood*

$$\log p(q | d) = \sum_i \log p(w_i | d)$$

where, $q = w_1 w_2 \dots w_n$

Document language model

- Retrieval problem $\approx$ Estimation of $p(w_i | d)$
- Common approach
 - Maximum likelihood estimation (MLE)



Parameter estimation

- General setting:
 - Given a (hypothesized & probabilistic) model that governs the random experiment
 - The model gives probability of any data $p(D|\theta)$ that depends on the parameter θ
 - Now, given actual sample data $X=\{x_1, \dots, x_n\}$, what can we say about the value of θ ?
- Intuitively, take our best guess of θ -- “best” means “best explaining/fitting the data”
- Generally an optimization problem

Maximum likelihood vs. Bayesian

- Maximum likelihood estimation

- “Best” means “data likelihood reaches maximum”

$$\hat{\theta} = \operatorname{argmax}_{\theta} P(X|\theta)$$

- Issue: small sample size

- Bayesian estimation

ML: Frequentist's point of view

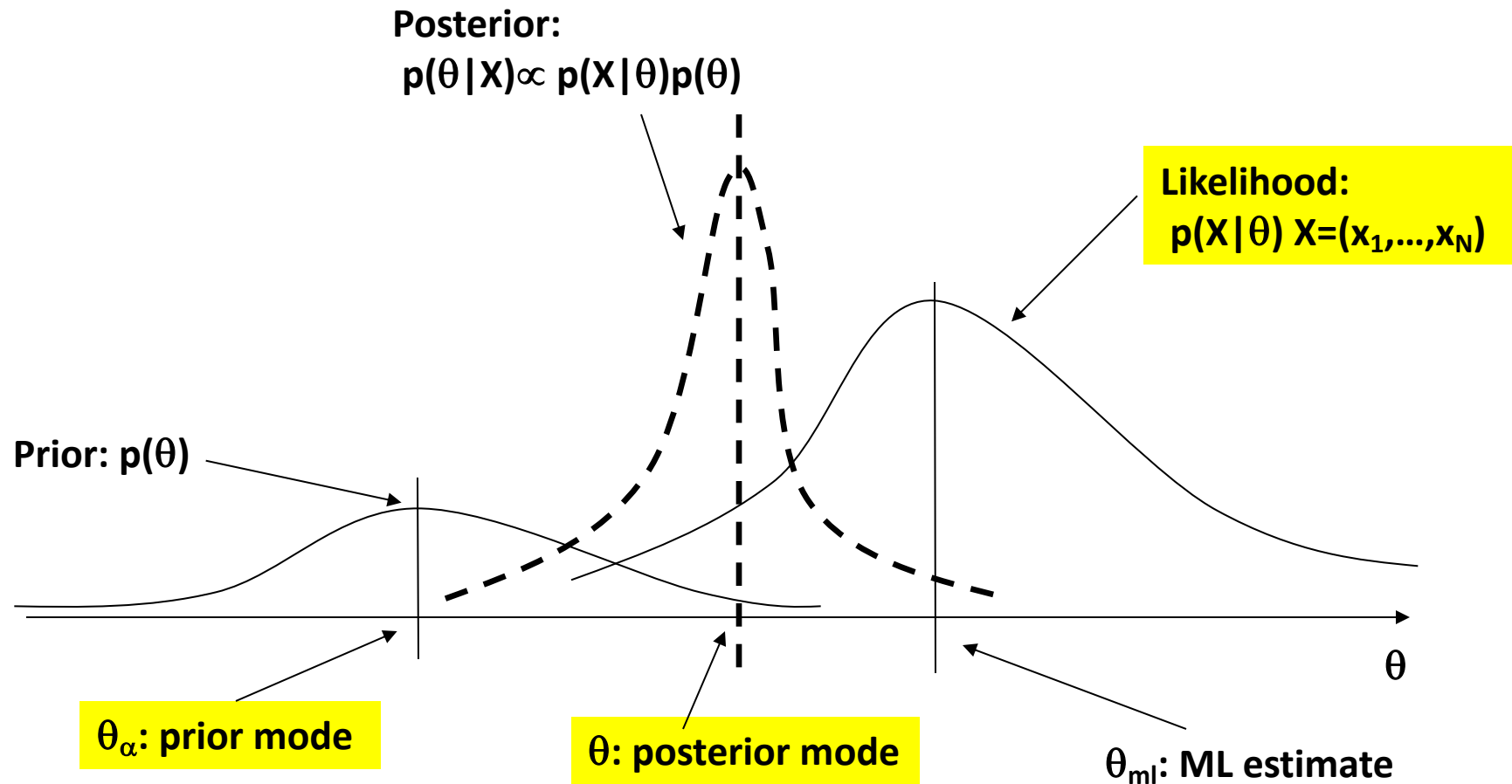
- “Best” means being consistent with our “prior” knowledge and explaining data well

$$\hat{\theta} = \operatorname{argmax}_{\theta} P(\theta|X) = \operatorname{argmax}_{\theta} P(X|\theta)P(\theta)$$

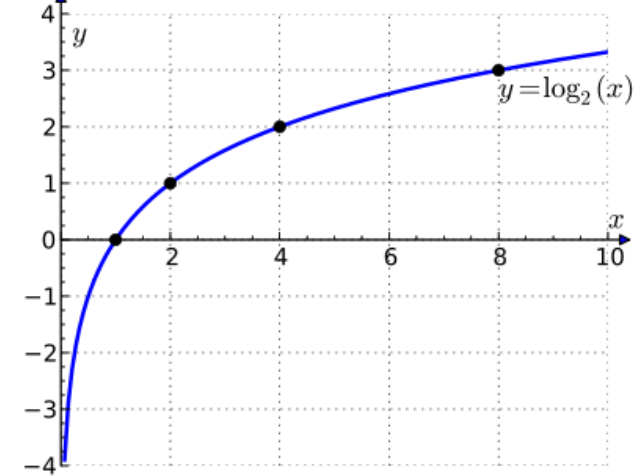
- A.k.a, Maximum a Posterior estimation
 - Issue: how to define prior?

MAP: Bayesian's point of view

Illustration of Bayesian estimation



Maximum likelihood estimation



- Data: a document d with counts $c(w_1), \dots, c(w_N)$
- Model: multinomial distribution $p(W|\theta)$ with parameters $\theta_i = p(w_i)$
- Maximum likelihood estimator: $\hat{\theta} = \operatorname{argmax}_{\theta} p(W|\theta)$

$$p(W|\theta) = \binom{N}{c(w_1), \dots, c(w_N)} \prod_{i=1}^N \theta_i^{c(w_i)} \propto \prod_{i=1}^N \theta_i^{c(w_i)} \Rightarrow \log p(W|\theta) = \sum_{i=1}^N c(w_i) \log \theta_i$$

$$\Rightarrow L(W, \theta) = \sum_{i=1}^N c(w_i) \log \theta_i + \lambda \left(\sum_{i=1}^N \theta_i - 1 \right)$$

Using Lagrange multiplier approach, we'll tune θ_i to maximize $L(W, \theta)$

$$\Rightarrow \frac{\partial L}{\partial \theta_i} = \frac{c(w_i)}{\theta_i} + \lambda \rightarrow \theta_i = -\frac{c(w_i)}{\lambda}$$

Set partial derivatives to zero

$$\Rightarrow \text{Since } \sum_{i=1}^N \theta_i = 1 \text{ we have } \lambda = -\sum_{i=1}^N c(w_i)$$

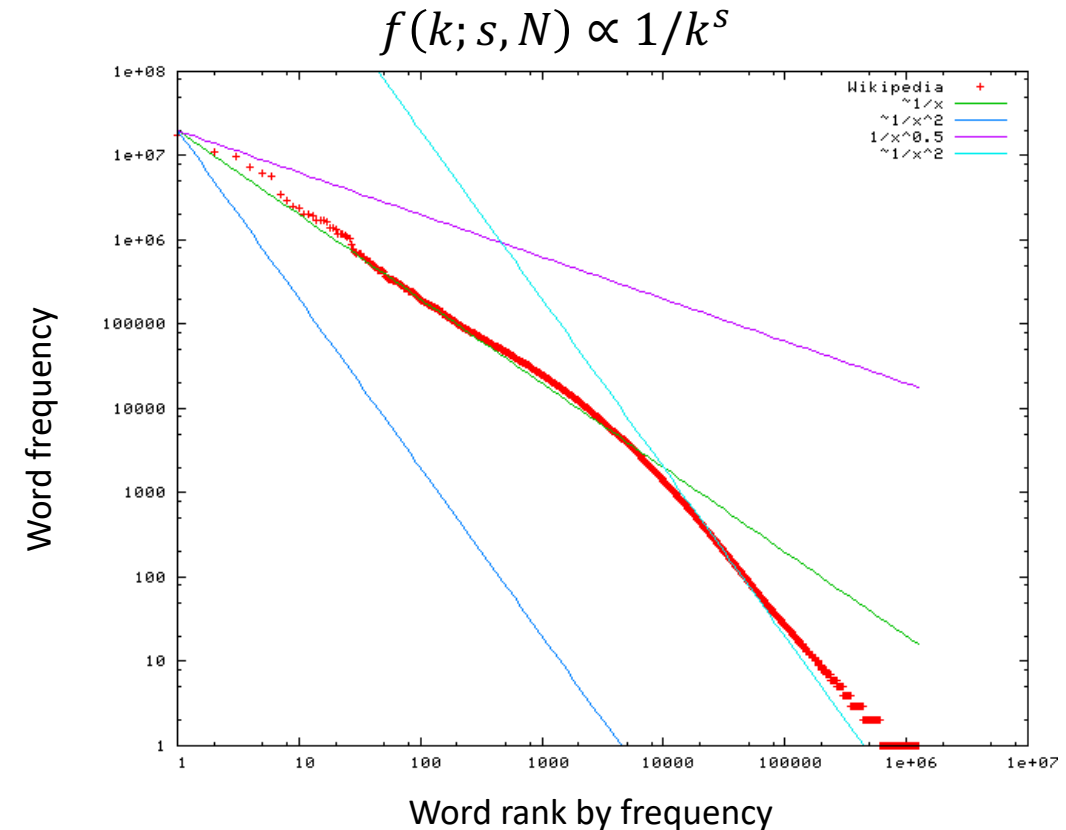
Requirement from probability

$$\Rightarrow \theta_i = \frac{c(w_i)}{\sum_{i=1}^N c(w_i)}$$

ML estimate

Problem with MLE

- Unseen events
 - There are 440K tokens on a large collection of Yelp reviews, but:
 - Only 30,000 unique words occurred
 - Only 0.04% of all possible bigrams occurred
 - This means any word/N-gram that does not occur in the collection has a zero probability with MLE!
 - No future queries/documents can contain those unseen words/N-grams

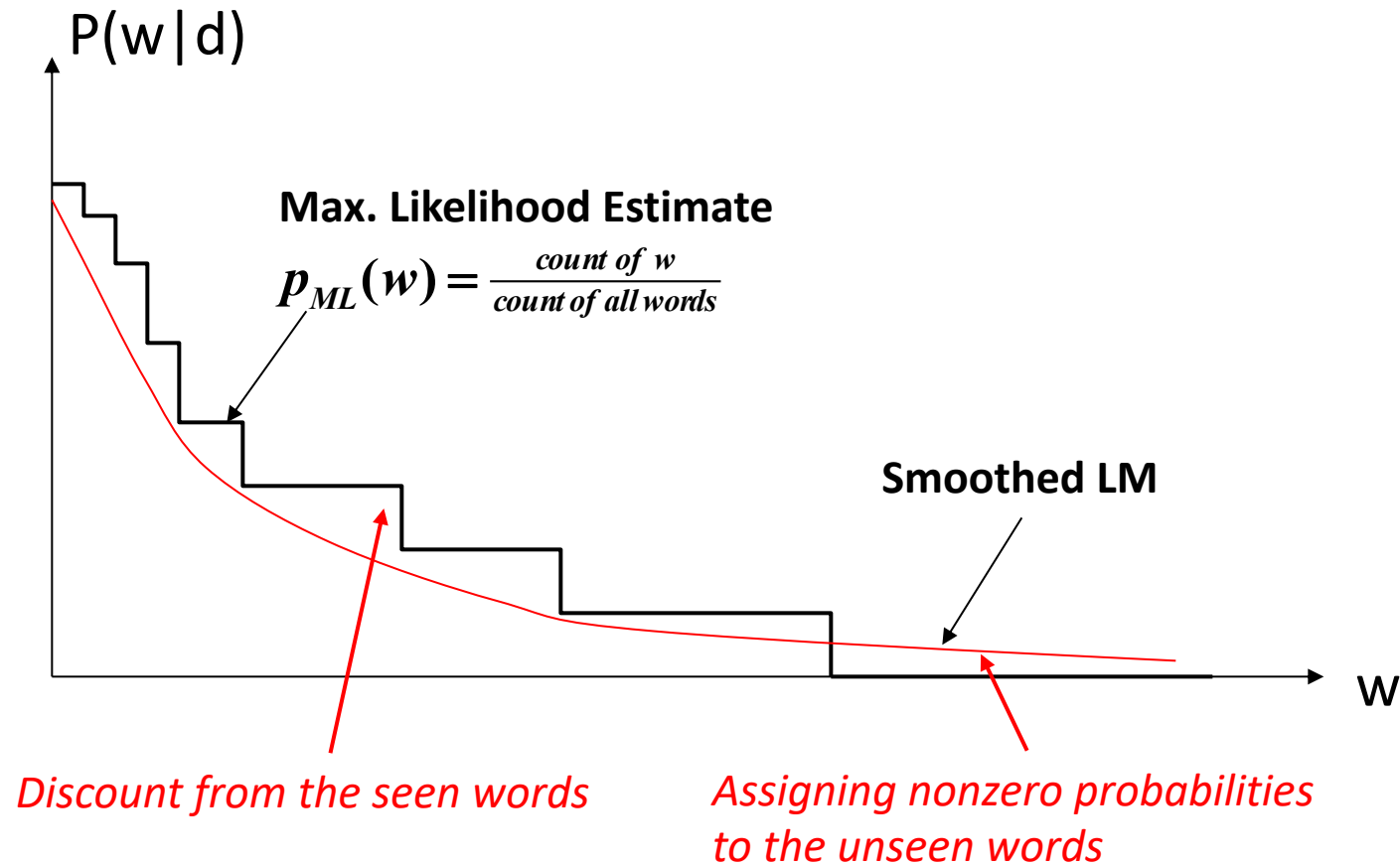


A plot of word frequency in Wikipedia (Nov 27, 2006)

Problem with MLE

- What probability should we give a word that has not been observed in the document?
 - $\log 0$?
- If we want to assign non-zero probabilities to such words, we'll have to discount the probabilities of observed words
- This is so-called “smoothing”

Illustration of language model smoothing

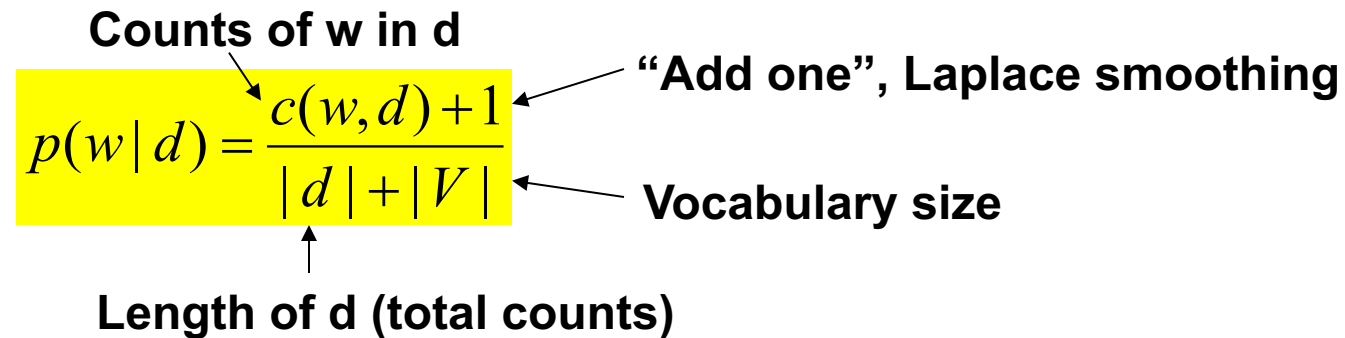


General idea of smoothing

- All smoothing methods try to
 1. Discount the probability of words seen in a document
 2. Re-allocate the extra counts such that unseen words will have a non-zero count

Smoothing methods

- Method 1: Additive smoothing
 - Add a constant δ to the counts of each word



The diagram shows the Laplace smoothing formula for word probability: $p(w|d) = \frac{c(w,d) + 1}{|d| + |V|}$. The entire formula is highlighted in yellow. Annotations include: an arrow from "Counts of w in d" pointing to the numerator term $c(w,d)$; an arrow from "“Add one”, Laplace smoothing" pointing to the $+1$ in the numerator; an arrow from "Vocabulary size" pointing to the $|V|$ in the denominator; and an arrow from "Length of d (total counts)" pointing to the $|d|$ in the denominator.

$$p(w|d) = \frac{c(w,d) + 1}{|d| + |V|}$$

Counts of w in d

“Add one”, Laplace smoothing

Vocabulary size

Length of d (total counts)

- Problems?
 - Hint: all words are equally important?

Add one smoothing for bigrams

Original:

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Smoothed:

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

After smoothing

- Giving too much to the unseen events without discrimination

Original:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Smoothed:

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Refine the idea of smoothing

- Should all unseen words get equal probabilities?
- We can use a reference model to discriminate unseen words

$$p(w|d) = \begin{cases} p_{seen}(w|d) & \text{if } w \text{ is seen in } d \\ \alpha_d p(w|REF) & \text{otherwise} \end{cases}$$

Discounted ML estimate

Reference language model

$$\alpha_d = \frac{1 - \sum_{w \text{ is seen}} p_{seen}(w|d)}{\sum_{w \text{ is unseen}} p(w|REF)}$$

Smoothing methods

- Method 2: Absolute discounting
 - Subtract a constant δ from the counts of each word

$$p(w|d) = \frac{\max(c(w;d) - \delta, 0) + \delta |d|_u p(w|REF)}{|d|}$$

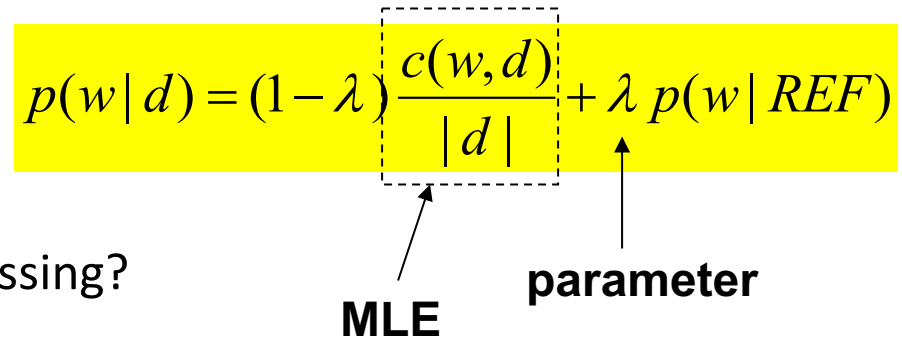
uniq words



- Problems?
 - Hint: varied document length?

Smoothing methods

- Method 3: Linear interpolation, Jelinek-Mercer
 - “Shrink” uniformly toward $p(w|REF)$

$$p(w|d) = (1 - \lambda) \frac{c(w, d)}{|d|} + \lambda p(w|REF)$$


The equation is displayed on a yellow rectangular background. A dashed rectangular box encloses the term $\frac{c(w, d)}{|d|}$. An arrow points from the label **MLE** below to the bottom of this dashed box. Another arrow points from the label **parameter** below to the λ coefficient in the second term of the equation.

- Problems?
 - Hint: what is missing?

Smoothing methods

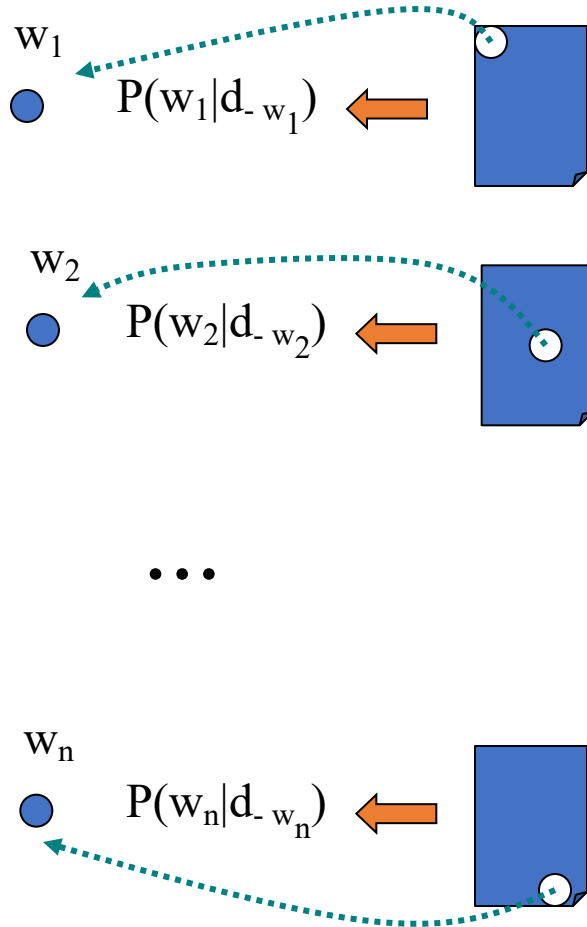
- Method 4: Dirichlet Prior/Bayesian
 - Assume pseudo counts $\mu p(w|REF)$

$$p(w|d) = \frac{c(w;d) + \mu p(w|REF)}{|d| + \mu} = \frac{|d|}{|d| + \mu} \frac{c(w,d)}{|d|} + \frac{\mu}{|d| + \mu} p(w|REF)$$

- Problems?

parameter

Estimating μ using leave-one-out [Zhai & Lafferty 02]



log-likelihood

$$L_{-1}(\mu | C) = \sum_{i=1}^N \sum_{w \in V} c(w, d_i) \log \left(\frac{c(w, d_i) - 1 + \mu p(w | C)}{|d_i| - 1 + \mu} \right)$$

Leave-one-out

Maximum Likelihood Estimator

$$\hat{\mu} = \operatorname{argmax}_{\mu} L_{-1}(\mu | C)$$

Understanding smoothing

	Query = "the algorithms for data mining"					
$p_{ML}(w d1):$	0.04	0.001	0.02	0.002	0.003	4.8×10^{-12}
$p_{ML}(w d2):$	0.02	0.001	0.01	0.003	0.004	2.4×10^{-12}

$p(\text{"algorithms"}|d1) = p(\text{"algorithms"}|d2)$
 $p(\text{"data"}|d1) < p(\text{"data"}|d2)$
 $p(\text{"mining"}|d1) < p(\text{"mining"}|d2)$

Intuitively, d2 should have a higher score, but $p(q|d1) > p(q|d2)$...

So we should make $p(\text{"the"})$ and $p(\text{"for"})$ **less different** for all docs, and smoothing helps to achieve this goal...

After smoothing with $p(w|d) = 0.1p_{DML}(w|d) + 0.9p(w|REF)$, $p(q|d1) < p(q|d2)$!

Query	= "the	algorithms	for	data	mining"
$P(w REF)$	0.2	0.00001	0.2	0.00001	0.00001
Smoothed $p(w d1):$	0.184	0.000109	0.182	0.000209	0.000309
Smoothed $p(w d2):$	0.182	0.000109	0.181	0.000309	0.000409

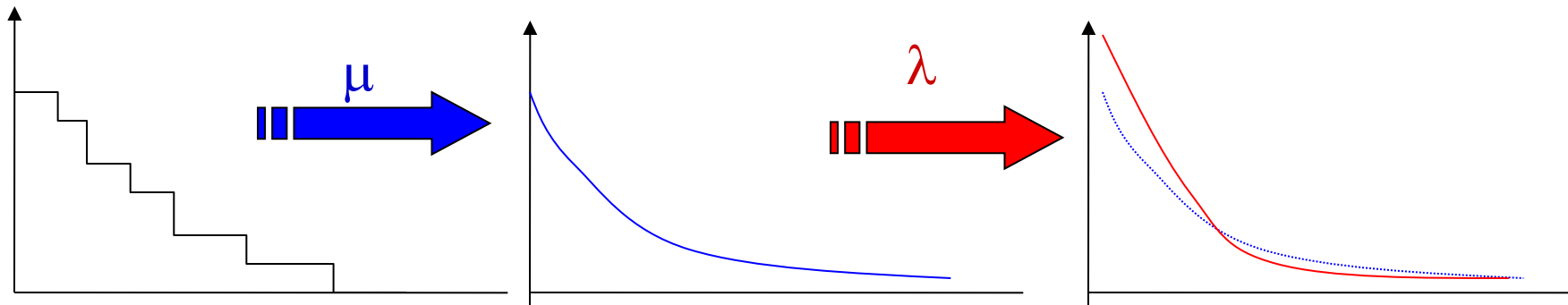
Two-stage smoothing [Zhai & Lafferty 02]

Stage-1

- Explain unseen words
- Dirichlet prior (Bayesian)

Stage-2

- Explain noise in query
- 2-component mixture



$$P(w|d) = (1-\lambda) \frac{c(w,d) + \underbrace{\mu p(w|C)}_{\text{Collection LM}}}{|d| + \underbrace{\mu}_{\text{User background model}}} + \lambda p(w|U)$$

Collection LM

User background model

Smoothing & TF-IDF weighting

Smoothed ML estimate

Reference language model

Retrieval formula using the general smoothing scheme

$$p(w|d) = \begin{cases} p_{\text{Seen}}(w|d) & \text{if } w \text{ is seen in } d \\ \alpha_d p(w|C) & \text{otherwise} \end{cases}$$

$$\alpha_d = \frac{1 - \sum_{w \text{ is seen}} p_{\text{Seen}}(w|d)}{\sum_{w \text{ is unseen}} p(w|C)}$$

↓

$$\log p(q|d) = \sum_{w \in V, c(w,q) > 0} c(w,q) \log p(w|d)$$

$$= \sum_{\substack{w \in V, c(w,d) > 0, \\ c(w,q) > 0}} c(w,q) \log p_{\text{Seen}}(w|d) + \sum_{w \in V, c(w,q) > 0, c(w,d) = 0} c(w,q) \log \alpha_d p(w|C)$$

$$= \sum_{\substack{w \in V, c(w,d) > 0, \\ c(w,q) > 0}} c(w,q) \log p_{\text{Seen}}(w|d) + \sum_{w \in V, c(w,q) > 0} c(w,q) \log \alpha_d p(w|C) - \sum_{w \in V, c(w,q) > 0, c(w,d) > 0} c(w,q) \log \alpha_d p(w|C)$$

$$= \sum_{\substack{w \in V, c(w,d) > 0, \\ c(w,q) > 0}} c(w,q) \log \frac{p_{\text{Seen}}(w|d)}{\alpha_d p(w|C)} + |q| \log \alpha_d + \sum_{w \in V, c(w,q) > 0} c(w,q) \log p(w|C)$$

↗ ↘

Key rewriting

Similar rewritings are very common when using probabilistic models for IR...

Understanding smoothing

$$\alpha_d = \frac{1 - \sum_{w \text{ is seen}} p_{\text{seen}}(w | d)}{\sum_{w \text{ is unseen}} p(w | REF)}$$

- Plug in the general smoothing scheme to the query likelihood retrieval formula, we obtain

$$\log p(q | d) = \sum_{w_i \in d \cap q} \left[\log \frac{p_{\text{seen}}(w_i | d)}{\alpha_d p(w_i | C)} \right] + |q| \log \alpha_d + \boxed{\sum_{w_i \in q} \log p(w_i | C)}$$

TF weighting

Doc length normalization
(longer doc is expected to have a smaller α_d)

IDF weighting

Ignore for ranking

- **Smoothing with $p(w|C) \approx$ TF-IDF + doc-length normalization**

Discussion

- Relation between the language model we learned today and LLM?

Summary

- How to estimate a language model
 - MLE / MAP
- General idea and different ways of smoothing
- Effect of smoothing

Readings

- [MRS] CH 12.